Integration Report

dotnet/skills → NotesApp Pipeline

Prepared: April 2026 | Scope: Test Quality + Performance Audit

What This Report Covers

Two skill clusters from the official **dotnet/skills** GitHub repository (github.com/dotnet/skills) have been evaluated against the NotesApp codebase. Everything else in that 87-skill repo is noise for this project. This report documents exactly which skills add value, how they integrate without creating overhead, and what changes concretely before and after.

The five criteria used to evaluate each skill:

- Better and more consistent coding sessions
- Token economy — no skill is loaded unless it earns its cost
- Better handling of the repository
- Better understanding of the codebase and alignment with best practices
- Knowing exactly when to include the human in the loop

Repository Landscape

The dotnet/skills repo contains **87 skills across 12 plugin categories**. The table below shows why most are irrelevant to NotesApp and which two clusters clear the bar.

Plugin	Skills	Relevance to NotesApp	Decision
dotnet-maui	8	Not a mobile project	No
dotnet-msbuild	14	No build perf problems identified	No
dotnet-template-engine	4	Not building templates	No
dotnet-nuget (CPM)	1	One-time migration, not ongoing	No
dotnet-ai (MCP)	5	Not building MCP servers	No
dotnet (core)	3	P/Invoke, scripting — not applicable	No
dotnet-upgrade	6	Relevant when moving to .NET 9 — not yet	Later
dotnet-aspnet	2	OpenTelemetry relevant only if tracing added	Later
dotnet-data (EF Core)	1	Overlaps with existing efcore-patterns skill	No
dotnet-experimental	4	Test quality — real gap in current pipeline	YES
dotnet-test	1	test-anti-patterns — stable, pragmatic	YES
dotnet-diag	1	analyzing-dotnet-performance — on-demand scan	YES

Cluster 1 — Test Quality (four skills)

Your existing skill stack (*csharp-coding-standards*, *efcore-patterns*, etc.) tells Claude how to write production code. None of those skills say anything about whether the tests written *for* that code are effective. That is the gap.

Skill	What it does	When triggered
exp-test-gap-analysis	Pseudo-mutation: asks 'if I broke this line, would any test catch it?'	After implementing a handler with complex branching
exp-mock-usage-analysis	Traces each mock setup through the production call path — finds dead setups and unreachable branches	When a handler test has more than 3 mocks
exp-test-smell-detection	Formal audit: 19 smell categories (assertion-free, eager test, mystery guest…)	Periodic suite health check, not per-feature
test-anti-patterns (stable)	Pragmatic: false confidence patterns, swallowed exceptions, always-true assertions	After writing tests, as part of every review

Where the Gap Shows Up Today — Concrete Examples

Example 1: Unreachable mock setup

From *RegisterDeviceCommandHandlerTests* — your CreateSut() pattern loads every mock setup for every test:

```
private RegisterDeviceCommandHandler CreateSut()
{
    _currentUserServiceMock
        .Setup(x => x.GetUserIdAsync(It.IsAny<CancellationToken>()))
        .ReturnsAsync(_currentUserId);

    _clockMock
        .Setup(x => x.UtcNow)
        .Returns(_utcNow);          // loaded for EVERY test

    return new RegisterDeviceCommandHandler(...);
}
```

Consider a validation-failure test:

```
[Fact]
public async Task Handle_WhenDeviceValidationFails_ReturnsDomainFailure()
{
    var sut = CreateSut();   // _clockMock.Setup runs here too
    var command = new RegisterDeviceCommand(invalidToken: "");

    var result = await sut.Handle(command, CancellationToken.None);

    result.IsFailed.Should().BeTrue();
}
```

exp-mock-usage-analysis traces the production execution path: validation fails → early return → ISystemClock.UtcNow is never called. The _clockMock setup is dead for this test. It loads, does nothing, and silently passes. You would not discover this until you changed the production code, removed the UtcNow call, and the test still passed when it should not.

Example 2: Survived mutation — test does not verify what it claims

A typical multi-branch handler:

```
public async Task<Result<NoteDetailDto>> Handle(UpdateNoteCommand command, CancellationToken ct)
{
    var userId = await _currentUserService.GetUserIdAsync(ct);
    var note   = await _noteRepository.GetByIdAsync(command.NoteId, ct);

    if (note is null || note.UserId != userId)
        return Result.Fail(new NotFoundError());        // early return A

    if (note.IsDeleted)
        return Result.Fail(new NotFoundError());        // early return B

    var domainResult = note.Update(command.Title, _clock.UtcNow);
    if (domainResult.IsFailed)
        return domainResult.ToResult<NoteDetailDto>();  // early return C

    // ... outbox + save
}
```

The 'updating deleted note' test:

```
[Fact]
public async Task Handle_WhenNoteIsDeleted_ReturnsNotFound()
{
    // arrange: note.IsDeleted = true
    var result = await sut.Handle(command, CancellationToken.None);

    result.IsFailed.Should().BeTrue();   // only assertion
}
```

exp-test-gap-analysis runs pseudo-mutation on early return B:

- *Mutation: remove the deleted check (early return B).* Handler proceeds to note.Update(). If the domain method also enforces IsDeleted, it returns a DomainFailure. result.IsFailed is still true — via a completely different code path. The test passes. The mutation **survived**.
- The assertion does not distinguish between NotFoundError from the deleted check vs DomainFailure from the domain method. This is a real test gap.

The recommended fix:

```
// Assert the error type, not just IsFailed
result.Errors.Should().ContainSingle(e => e is NotFoundError);

// OR assert what did NOT happen
_noteRepositoryMock.Verify(
    x => x.Update(It.IsAny<Note>()),
    Times.Never);
```

Example 3: Assertion quality — loose clock assertion

From your Application.Tests assertion patterns:

```
// Precise — good
dto.Title.Should().Be(command.Title);
dto.CreatedAtUtc.Should().BeOnOrAfter(before);
persisted!.UserId.Should().Be(userId);

// Loose — quality signal
dto.UpdatedAtUtc.Should().BeCloseTo(DateTime.UtcNow, TimeSpan.FromMinutes(1));
```

The 1-minute tolerance exists because of clock drift between test setup and assertion. But your codebase already injects ISystemClock — you control the clock. **exp-assertion-quality** would flag BeCloseTo(... FromMinutes(1)) as a weak assertion and suggest:

```
_clockMock.Setup(x => x.UtcNow).Returns(_fixedUtcNow);

// Then:
dto.UpdatedAtUtc.Should().Be(_fixedUtcNow);  // exact, no tolerance
```

How Integration Works Without Creating Noise

These skills are **not added to the default task flow**. Token cost is zero unless explicitly triggered.

Current flow (unchanged):

```
pre-task → codebase-researcher → docs-researcher → plan → implement → simplify
```

Proposed additions — targeted, not always-on:

```
After implement + simplify:
  ■■■ If new tests were written:
        ■■■ test-anti-patterns        (stable, quick pass — Critical/High only)
        ■■■ exp-mock-usage-analysis   (only if handler test uses >3 mocks)

Periodic (not per-task):
  ■■■ exp-test-smell-detection   (suite health check, once per major feature)
  ■■■ exp-test-gap-analysis      (handlers with complex branching / multiple early returns)
```

The CLAUDE.md addition is surgical:

```
## Tests — quality gate

After writing tests for a new handler:
- Run `test-anti-patterns` on the new test file (Critical/High only)
- Run `exp-mock-usage-analysis` if the handler test uses more than 3 mocks

For handlers with complex branching (multiple early returns, domain logic, outbox):
- Run `exp-test-gap-analysis` against the handler + its test file
- Not mandatory for every handler — use judgment based on complexity
```

Before vs. After — What Actually Changes

Aspect	Before	After
Dead mock detection	Discovered only when production code changes and tests don't break as expected	Caught proactively when the test is written — 'this setup is unreachable for this test path'
Error type precision	Tests assert IsFailed — passes even if wrong error type returned	Gap analysis flags: 'you have no test that distinguishes NotFoundError from DomainFailure here'
Clock assertion quality	BeCloseTo(... FromMinutes(1)) silently accepted	Flagged: 'you control the clock via ISystemClock — use exact assertion'
Naming consistency	Mixed snake_case/PascalCase across Application.Tests and Api.IntegrationTests	test-anti-patterns flags as Low-severity, surfaces for conscious decision
Mutation survivors	UpdateNoteCommandHandlerTests covers deleted path but assertion doesn't distinguish which failure path triggered	exp-test-gap-analysis surfaces 'removing the deleted check doesn't break your test — caught downstream'; you decide if acceptable
Over-mocking in shared setup	CreateSut() loads all setups for all tests, including ones irrelevant to the specific test	exp-mock-usage-analysis identifies which setups are dead per test, suggests per-test setup for specificity

Cluster 2 — Performance Anti-Pattern Audit

analyzing-dotnet-performance (dotnet-diag plugin) is a scan tool, not a style guide. It reads code and flags ~50 specific patterns by severity.

Severity	Examples
Critical (deadlocks, >10x regression)	.Result or .Wait() on async, async void, Thread.Sleep in async code
Moderate (2–10x improvement opportunity)	string.Format in hot loops, ToList() before Where(), Regex compiled per-call instead of cached, List where IEnumerable avoids copy
Info (applicable pattern, non-critical path)	LINQ over-chaining, Dictionary with default equality comparer where custom would help

How It Differs from Your Existing Skills

Your existing *csharp-type-design-performance* and *csharp-coding-standards* are **prescriptive** — they tell Claude how to write new code. *analyzing-dotnet-performance* is **diagnostic** — it scans existing code for violations.

Skill	Orientation	Example output
csharp-type-design-performance	Prescriptive — write this way	'When writing a new method, defer .ToList() to the end'
analyzing-dotnet-performance	Diagnostic — scan existing code	'In NotesQueryHandler.cs line 47, .ToList() before .Where() materialises the entire collection. Moderate finding.'

When to Use It (and When Not To)

Use it for:

- A dedicated perf pass when a feature area shows slowness
- Pre-release audit of a handler that touches large or unbounded collections
- Reviewing the Worker — outbox processing iterates collections

Do not add it to the default task flow. It is a point-in-time scan, not a per-commit gate. The CLAUDE.md entry:

```
## Performance audit

Run `analyzing-dotnet-performance` when:
- A handler iterates or queries collections of unbounded size
- The Worker outbox processing loop is being modified
- A query handler is identified as slow in production

Do not run it as a default step — it is a targeted diagnostic, not a style gate.
```

Before vs. After

Before	After
Performance issues caught only when someone notices slowness in production	Systematic scan available on-demand with structured Critical/Moderate/Info triage
csharp-type-design-performance prevents new anti-patterns but does not scan existing code	Existing code can be audited against the same standards on demand
'Is this query handler efficient?' answered by reading code manually	Answered by running the skill and receiving a structured severity report with file/line references

What Does Not Get Added, and Why

Explicit about the line:

dotnet/skills area	Decision	Reason
MAUI (8 skills)	No	Not a mobile project
MSBuild (14 skills)	No	No build performance problems identified
Template engine (4 skills)	No	Not building templates
dotnet-upgrade (6 skills)	Not yet	Relevant when upgrading to .NET 9 — flag then
dotnet-aspnet / OpenTelemetry	Later	Relevant only if tracing infrastructure is added
dotnet-data / EF Core optimising	No	Already covered by efcore-patterns — overlaps without adding new ground
dotnet-ai / MCP (5 skills)	No	Not building AI infrastructure
convert-to-cpm	No	One-time migration — not an ongoing session concern
exp-simd-vectorization	No	Not applicable to a web API notes app
exp-test-maintainability	No	test-anti-patterns already covers the relevant overlap; adding both scans the same ground twice

Summary: Net Change to the Pipeline

Two additions, both conditional:

1. Test quality gate — test-anti-patterns + exp-mock-usage-analysis + exp-test-gap-analysis

Triggered after writing tests for handlers with branching logic. Not in every task. The CLAUDE.md entry makes it explicit when to trigger each one.

2. Performance audit — analyzing-dotnet-performance

Triggered on-demand for perf-sensitive areas. Not in the default flow.

The *exp-test-smell-detection* and *exp-assertion-quality* skills are worth keeping available but are better used as a periodic audit (every few features) rather than per-task triggers — no CLAUDE.md entry needed for them.

Nothing else from the 87-skill repo is relevant to this codebase right now.